



Awaiters and Awaitables

MATEUSZ PUSZ



20
26



So You Have an Async API

WHAT YOU HAVE

```
void read_temperature(sensor& s,  
                      std::function<void(int)> on_done);
```

WHAT YOU WANT TO WRITE

```
task<void> monitor(sensor& s)  
{  
    int t = co_await read_temperature(s);  
    log(t);  
}
```

So You Have an Async API

WHAT YOU HAVE

```
void read_temperature(sensor& s,  
                      std::function<void(int)> on_done);
```

WHAT YOU WANT TO WRITE

```
task<void> monitor(sensor& s)  
{  
    int t = co_await read_temperature(s);  
    log(t);  
}
```

Every codebase has a boundary like this

- An OS timer, or a hardware interrupt
- A thread pool completion callback
- A legacy `std::future<T>` somebody else owns
- Your company's in-house async framework
- A third-party C library that only speaks function pointers

So You Have an Async API

WHAT YOU HAVE

```
void read_temperature(sensor& s,  
                      std::function<void(int)> on_done);
```

WHAT YOU WANT TO WRITE

```
task<void> monitor(sensor& s)  
{  
    int t = co_await read_temperature(s);  
    log(t);  
}
```

Every codebase has a boundary like this

- An OS timer, or a hardware interrupt
- A thread pool completion callback
- A legacy `std::future<T>` somebody else owns
- Your company's in-house async framework
- A third-party C library that only speaks function pointers

The thing that connects the top to the bottom one is called an awaiter.
Writing one is the entire subject of this talk.

This Is What the Bridge Buys You

CALLBACKS

```
void do_read()
{
    auto self = shared_from_this();
    socket_.async_read_some(asio::buffer(data_),
                           [this, self](error_code ec,
                                         size_t n) {
                               if (!ec) do_write(n);
                           });
}

void do_write(size_t n)
{
    auto self = shared_from_this();
    asio::async_write(socket_,
                     asio::buffer(data_.data(), n),
                     [this, self](error_code ec, size_t) {
                         if (!ec) do_read();
                     });
}
```

This Is What the Bridge Buys You

CALLBACKS

```
void do_read()
{
    auto self = shared_from_this();
    socket_.async_read_some(asio::buffer(data_),
                           [this, self](error_code ec,
                                         size_t n) {
                               if (!ec) do_write(n);
                           });
}

void do_write(size_t n)
{
    auto self = shared_from_this();
    asio::async_write(socket_,
                     asio::buffer(data_.data(), n),
                     [this, self](error_code ec, size_t) {
                         if (!ec) do_read();
                     });
}
```

co_await

```
awaitable<void> session(tcp::socket socket)
{
    std::array<std::byte, 1024> data;
    while(true) {
        auto [ec, n] =
            co_await socket.async_read_some(asio::buffer(data),
                                           asio::as_tuple);

        if (n == 0) break;

        auto [ec2, nw] =
            co_await asio::async_write(socket,
                                       asio::buffer(data, n),
                                       asio::as_tuple);

        if (nw != n) break;
    }
}
```

Somebody Had to Write That

```
auto [ec, n] = co_await socket.async_read_some(asio::buffer(data), asio::as_tuple);
```

Somebody Had to Write That

```
auto [ec, n] = co_await socket.async_read_some(asio::buffer(data), asio::as_tuple);
```

For that one line to work, someone wrote code that

- **Suspends** the coroutine when the read starts
- **Hands the coroutine's handle** to the I/O completion callback
- **Resumes** the coroutine when the read finishes
- **Delivers** `ec` and `n` as **the value** of the `co_await` expression

Somebody Had to Write That

```
auto [ec, n] = co_await socket.async_read_some(asio::buffer(data), asio::as_tuple);
```

For that one line to work, someone wrote code that

- **Suspends** the coroutine when the read starts
- **Hands the coroutine's handle** to the I/O completion callback
- **Resumes** the coroutine when the read finishes
- **Delivers** `ec` and `n` as **the value** of the `co_await` expression

Asio's authors wrote that awaiter. Nobody wrote one for your sensor API.

This Is Not a `task<T>` Talk

Writing a coroutine `return type` from scratch means understanding

- `promise_type` and its mandatory interface
- `std::coroutine_traits`
- `std::coroutine_handle`
- Eager vs. lazy task semantics
- exception plumbing
- `sync_await`
- Symmetric Control Transfer
- Heap allocation, allocation elision, custom allocators

This Is Not a `task<T>` Talk

GET ONE FROM A LIBRARY

- `asio::awaitable<T>`
- `cppcoro::task<T>`
- `unifex::task<T>`

This Is Not a `task<T>` Talk

GET ONE FROM A LIBRARY

- `asio::awaitable<T>`
- `cppcoro::task<T>`
- `unifex::task<T>`

OR FROM THE STANDARD

- `std::generator<T>` (C++23, here today)
- `std::execution::task<T>` (C++26)

This Is Not a `task<T>` Talk

GET ONE FROM A LIBRARY

- `asio::awaitable<T>`
- `cppcoro::task<T>`
- `unifex::task<T>`

OR FROM THE STANDARD

- `std::generator<T>` (C++23, here today)
- `std::execution::task<T>` (C++26)

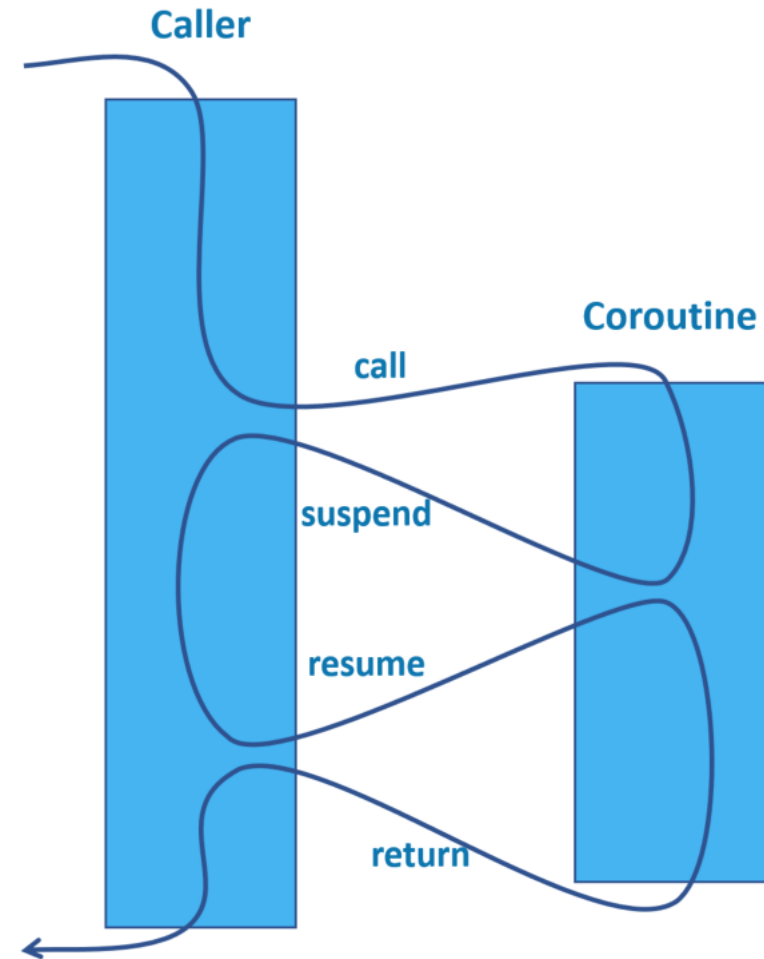
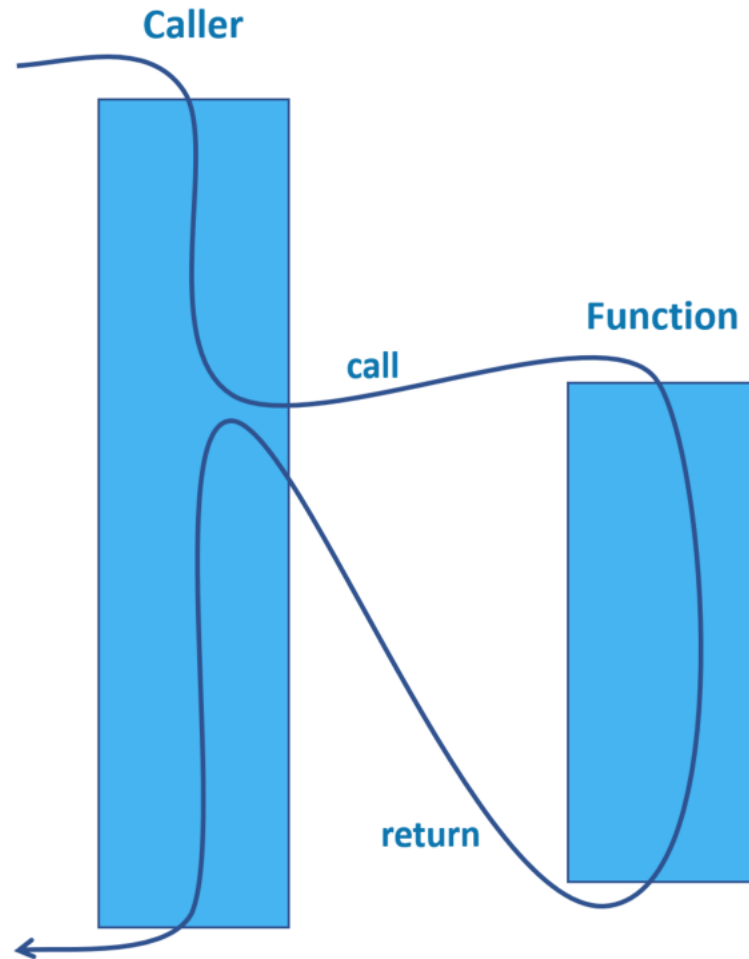
That is a two-day workshop. Today `task<T>` is given, and we write the piece no library can write for you.

THE AWAITER PROTOCOL

A Coroutine

A coroutine is a function that can suspend its execution and be resumed later.

A Coroutine



A Coroutine

A function becomes a coroutine when its body contains any of

```
co_await expr;           // suspend until a result is ready
co_yield value;          // suspend, handing a value out
co_return value;         // complete execution
```

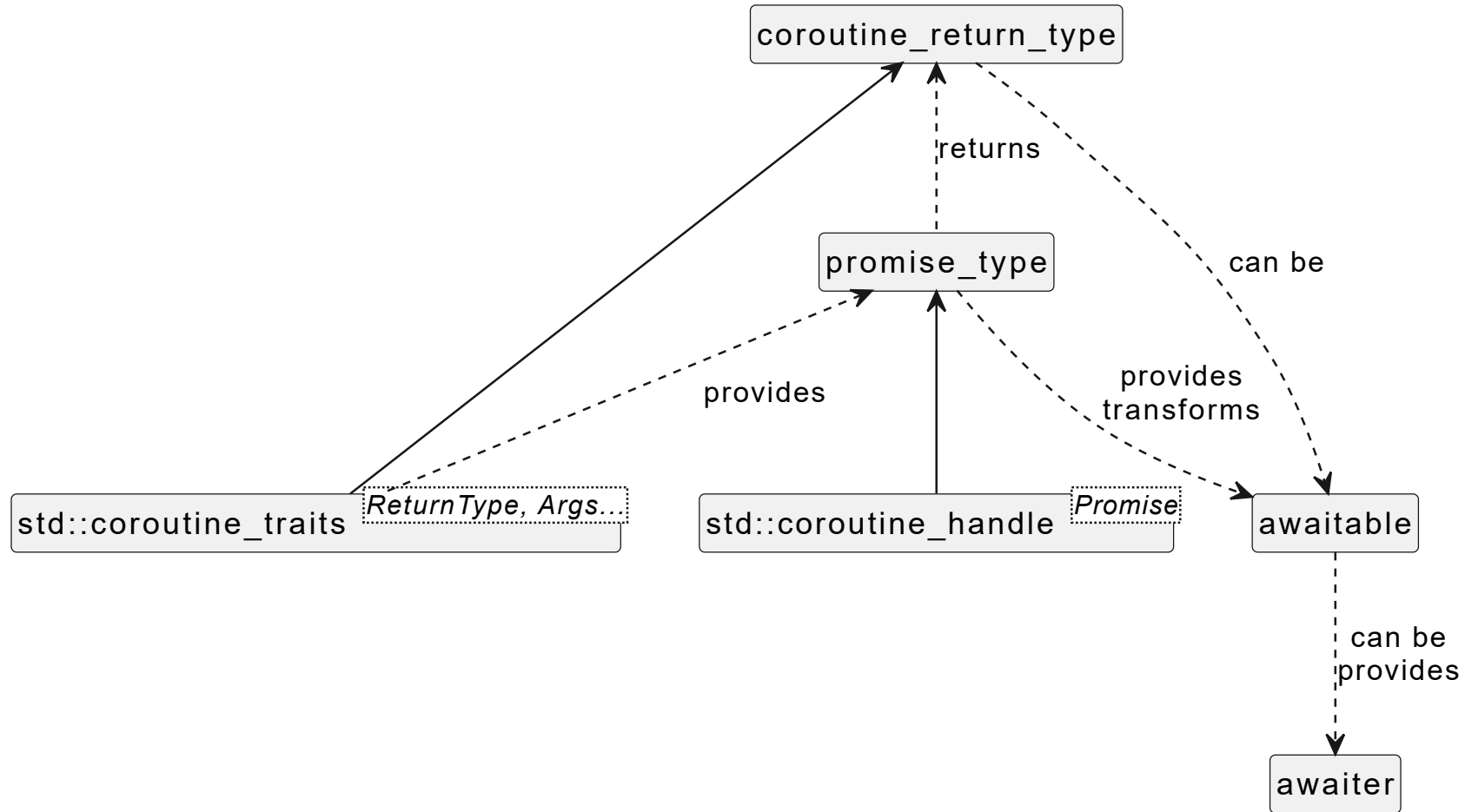
A Coroutine

A function becomes a coroutine when its body contains any of

```
co_await expr;      // suspend until a result is ready  
co_yield value;     // suspend, handing a value out  
co_return value;    // complete execution
```

Those keywords must appear directly in the function's body. Having them in subfunctions does not count.

Coroutine Engine Types



std::coroutine_handle<>

```
template<class Promise = void>  
struct coroutine_handle;
```

- A **non-owning**, lightweight handle to a suspended coroutine
 - trivial destructor

std::coroutine_handle<>

```
template<class Promise = void>
struct coroutine_handle;
```

- A **non-owning**, lightweight handle to a suspended coroutine
 - trivial destructor
- The two operations an **awaiter needs**

```
h.resume();    // resumes the suspended coroutine
h.done();      // true if the coroutine is at its final suspend point
```

std::coroutine_handle<>

```
template<class Promise = void>
struct coroutine_handle;
```

- A **non-owning**, lightweight handle to a suspended coroutine
 - trivial destructor
- The two operations an **awaiter needs**

```
h.resume();    // resumes the suspended coroutine
h.done();      // true if the coroutine is at its final suspend point
```

- The operation we **should not use**

```
h.destroy();   // destroys the coroutine
```

- more on that later

std::coroutine_handle<>

```
template<class Promise = void>
struct coroutine_handle;
```

IN AWAITER CODE

- Use `std::coroutine_handle<>` (type erased) **by default**

std::coroutine_handle<>

```
template<class Promise = void>
struct coroutine_handle;
```

IN AWAITER CODE

- Use `std::coroutine_handle<>` (type erased) **by default**
- Reach for `std::coroutine_handle<P>` when you need something **from the awaiting coroutine's promise**
 - its scheduler, its stop token, ...

std::coroutine_handle<>

```
template<class Promise = void>
struct coroutine_handle;
```

IN AWAITER CODE

- Use `std::coroutine_handle<>` (type erased) **by default**
- Reach for `std::coroutine_handle<P>` when you need something **from the awaiting coroutine's promise**
 - its scheduler, its stop token, ...

Naming one concrete promise type is what locks your awaiter to one coroutine type.

An Awaiter Is Three Functions

```
struct awaiter {  
    bool await_ready();  
    auto await_suspend(std::coroutine_handle<> h);  
    T    await_resume();  
};
```

An Awaiter Is Three Functions

```
struct awaiter {  
    bool await_ready();  
    auto await_suspend(std::coroutine_handle<> h);  
    T      await_resume();  
};
```

- `await_ready()`: is the result already here?

An Awaiter Is Three Functions

```
struct awaiter {  
    bool await_ready();  
    auto await_suspend(std::coroutine_handle<> h);  
    T      await_resume();  
};
```

- `await_ready()`: is the result already here?
- `await_suspend(h)`: the coroutine is now suspended. Arrange for it to come back.

An Awaiter Is Three Functions

```
struct awaiter {  
    bool await_ready();  
    auto await_suspend(std::coroutine_handle<> h);  
    T      await_resume();  
};
```

- `await_ready()`: is the result already here?
- `await_suspend(h)`: the coroutine is now suspended. Arrange for it to come back.
- `await_resume()`: what is the value of the `co_await` expression?

An Awaiter Is Three Functions

```
struct awaiter {  
    bool await_ready();  
    auto await_suspend(std::coroutine_handle<> h);  
    T      await_resume();  
};
```

- `await_ready()`: is the result already here?
- `await_suspend(h)`: the coroutine is now suspended. Arrange for it to come back.
- `await_resume()`: what is the value of the `co_await` expression?

`await_suspend` and `await_resume` are callbacks: they run after the coroutine suspended and after it resumed.

await_ready

```
bool await_ready();
```

await_ready

```
bool await_ready();
```

- Called **first**, before any suspension happens
- **true**: the result is already here, **skip suspension entirely**
- **false**: suspend the coroutine

await_ready

```
bool await_ready();
```

- Called **first**, before any suspension happens
- **true**: the result is already here, **skip suspension entirely**
- **false**: suspend the coroutine

This is an optimization hook. Suspension means storing live locals and a resume point, then a round trip back through whoever resumes you.

await_suspend

```
auto await_suspend(std::coroutine_handle<> h);
```

await_suspend

```
auto await_suspend(std::coroutine_handle<> h);
```

- Called **after the coroutine has suspended**
- **h** is a handle to that suspended coroutine
- Responsibility: ensure that coroutine is **resumed or destroyed, exactly once**

await_suspend

```
auto await_suspend(std::coroutine_handle<> h);
```

- Called **after the coroutine has suspended**
- **h** is a handle to that suspended coroutine
- Responsibility: ensure that coroutine is **resumed or destroyed, exactly once**

THREE LEGAL RETURN TYPES

RETURN TYPE	MEANING
<code>void</code>	Stay suspended. Return control to the caller.
<code>bool</code>	false = resume right now; true = stay suspended.
<code>std::coroutine_handle<Z></code>	Resume coroutine Z , not the caller (symmetric control transfer).

await_suspend

```
auto await_suspend(std::coroutine_handle<> h);
```

- Called **after the coroutine has suspended**
- **h** is a handle to that suspended coroutine
- Responsibility: ensure that coroutine is **resumed or destroyed, exactly once**

If **await_suspend** throws, the coroutine is resumed and the exception is rethrown at the **co_await** site. **await_resume()** is not called.

await_resume

```
T await_resume();
```

await_resume

```
T await_resume();
```

- Called **when the coroutine resumes**
- Its return value **is** the value of the **co_await** expression

```
int value = co_await my_awaiter{};  
//          ^^^^^^^^^^^^^^^^^^^^^^^  
//          this is await_resume()'s return value
```

await_resume

```
T await_resume();
```

- Called **when the coroutine resumes**
- Its return value **is** the value of the **co_await** expression

```
int value = co_await my_awaiter{};  
//          ^^^^^^^^^^^^^^^^^^^^^^^^^^^  
//          this is await_resume()'s return value
```

- The result of **initial_suspend()** / **final_suspend()** **is always discarded**
 - should return **void**

await_resume

```
T await_resume();
```

- Called **when the coroutine resumes**
- Its return value **is** the value of the **co_await** expression

```
int value = co_await my_awaiter{};  
//          ^^^^^^^^^^^^^^^^^^^^^^^  
//          this is await_resume()'s return value
```

- The result of **initial_suspend()** / **final_suspend()** **is always discarded**
 - should return **void**

If the async operation finished with an error, the function should rethrow.

What `co_await` Actually Compiles To

```
auto result = co_await expr;
```

What `co_await` Actually Compiles To

```
auto result = co_await expr;
```

```
auto awaiter = get_awaiter(get_awaitable(expr));    // we come back to this
if (!awaiter.await_ready()) {
    // suspend coroutine; write locals and the resume point into the coroutine frame
    awaiter.await_suspend(this_coro_handle);        // hand off to the async system
    if(/* still suspended */)
        return;                                    // control goes to the caller/resumer
}
label_resume:                                     // reached after h.resume() is called
auto result = awaiter.await_resume();              // pick up the result
```

What `co_await` Actually Compiles To

```
auto result = co_await expr;
```

```
auto awaiter = get_awaiter(get_awaitable(expr));    // we come back to this
if (!awaiter.await_ready()) {
    // suspend coroutine; write locals and the resume point into the coroutine frame
    awaiter.await_suspend(this_coro_handle);        // hand off to the async system
    if(/* still suspended */)
        return;                                    // control goes to the caller/resumer
}
label_resume:                                     // reached after h.resume() is called
auto result = awaiter.await_resume();              // pick up the result
```

return here means "return to whoever resumed us" (caller or resumer), not "return from the coroutine".

The Trivial Awaitables

`std::suspend_never`

```
struct suspend_never {  
    constexpr bool await_ready() const noexcept  
    { return true; }  
    constexpr void await_suspend(  
        std::coroutine_handle<>) const noexcept {}  
    constexpr void await_resume() const noexcept {}  
};
```

Never suspends

The Trivial Awaitables

`std::suspend_never`

```
struct suspend_never {  
    constexpr bool await_ready() const noexcept  
    { return true; }  
    constexpr void await_suspend(  
        std::coroutine_handle<>) const noexcept {}  
    constexpr void await_resume() const noexcept {}  
};
```

Never suspends

`std::suspend_always`

```
struct suspend_always {  
    constexpr bool await_ready() const noexcept  
    { return false; }  
    constexpr void await_suspend(  
        std::coroutine_handle<>) const noexcept {}  
    constexpr void await_resume() const noexcept {}  
};
```

Always suspends, and never resumes

BUILDING A REAL AWAITER

The Goal

```
task<void> foo()
{
    using namespace std::chrono_literals;

    std::cout << "About to sleep\n";
    co_await sleep_for(1s);
    std::cout << "Done!\n";
}
```

Let the Compiler Guide Us

```
template<typename D>
struct sleep_for {
    D duration;
};
```

Let the Compiler Guide Us

```
template<typename D>
struct sleep_for {
    D duration;
};
```

```
task<void> foo()
{
    co_await sleep_for(1s);
}
```

Let the Compiler Guide Us

```
template<typename D>
struct sleep_for {
    D duration;
};
```

```
task<void> foo()
{
    co_await sleep_for(1s);
}
```

error: no member named 'await_ready' in 'sleep_for<std::chrono::duration<long int>>'

Let the Compiler Guide Us

```
template<typename D>
struct sleep_for {
    D duration;
};
```

```
task<void> foo()
{
    co_await sleep_for(1s);
}
```

error: no member named 'await_ready' in 'sleep_for<std::chrono::duration<long int>>'

The compiler is telling us exactly what to implement. Let's give it what it wants, one function at a time.

Step 1: await_ready

```
template<typename D>
struct sleep_for {
    D duration;

    bool await_ready() const noexcept { return duration <= D::zero(); }
};
```

Step 1: `await_ready`

```
template<typename D>
struct sleep_for {
    D duration;

    bool await_ready() const noexcept { return duration <= D::zero(); }
};
```

`co_await sleep_for(0s)` should cost nothing - no suspension.

Step 2: await_suspend

```
template<typename D>
struct sleep_for {
    D duration;

    bool await_ready() const noexcept { return duration <= D::zero(); }

    void await_suspend(std::coroutine_handle<> h)
    {
        std::this_thread::sleep_for(duration);    // wait
        h.resume();                               // then resume the coroutine
    }
};
```

Step 2: await_suspend

```
template<typename D>
struct sleep_for {
    D duration;

    bool await_ready() const noexcept { return duration <= D::zero(); }

    void await_suspend(std::coroutine_handle<> h)
    {
        std::this_thread::sleep_for(duration);    // wait
        h.resume();                               // then resume the coroutine
    }
};
```

This blocks the calling thread, which is a terrible timer and a perfectly good teaching device.

Step 3: await_resume

```
template<typename D>
struct sleep_for {
    D duration;

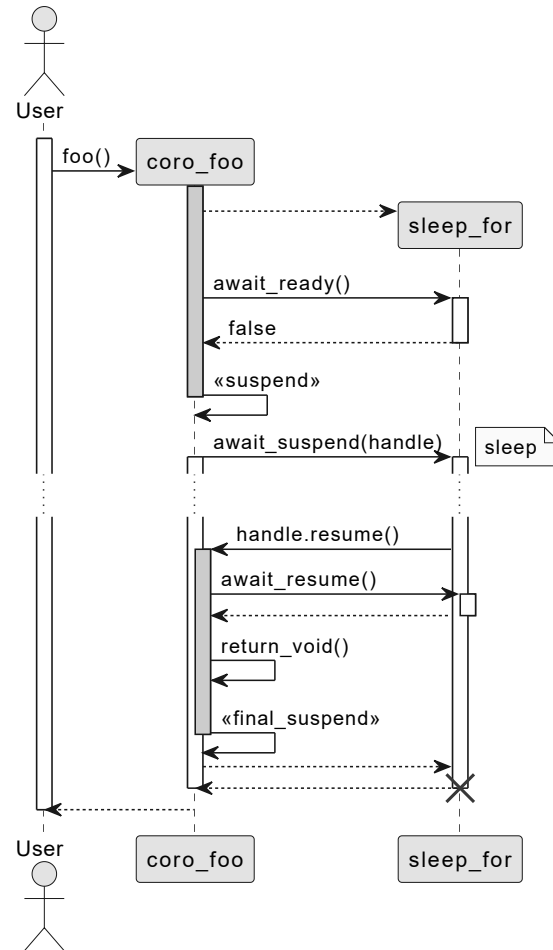
    bool await_ready() const noexcept { return duration <= D::zero(); }

    void await_suspend(std::coroutine_handle<> h)
    {
        std::this_thread::sleep_for(duration);
        h.resume();
    }

    void await_resume() noexcept {}    // sleep_for produces no value
};
```

That is a complete, working awaiter.

How Does It Flow?



SYMMETRIC CONTROL TRANSFER

The Problem with Resuming Inside `await_suspend`

```
void await_suspend(std::coroutine_handle<> h)
{
    std::this_thread::sleep_for(duration);
    h.resume();    // ← an ordinary function call. It grows the stack.
}
```

The Problem with Resuming Inside `await_suspend`

```
void await_suspend(std::coroutine_handle<> h)
{
    std::this_thread::sleep_for(duration);
    h.resume();    // ← an ordinary function call. It grows the stack.
}
```

Every `h.resume()` inside `await_suspend` adds a frame that is never popped until the innermost coroutine finishes. Stack depth grows with the length of the await chain, and a long chain overflows the stack.

Regular Call vs Symmetric Control Transfer

REGULAR CALL

```
int foo() { return 42; }  
int bar() { return foo() + 23; }
```

```
int main()  
{  
    int res = bar();  
}
```

- **Stack grows** with every new function invocation

Regular Call vs Symmetric Control Transfer

REGULAR CALL

```
int foo() { return 42; }  
int bar() { return foo() + 23; }
```

```
int main()  
{  
    int res = bar();  
}
```

- **Stack grows** with every new function invocation

SYMMETRIC CONTROL TRANSFER

```
int foo() { return 42; }  
auto bar() { return []{ return foo() + 23; } }
```

```
int main()  
{  
    auto continuation = bar();  
    int res = continuation();  
}
```

- **Stack is flat**
- Some additional machinery needed to handle the control transfer

Symmetric Control Transfer

Instead of calling `h.resume()` inside `await_suspend`, return `h` or `false` from it. The coroutine machinery resumes it at the caller's frame level, so the stack stays flat.

Symmetric Control Transfer

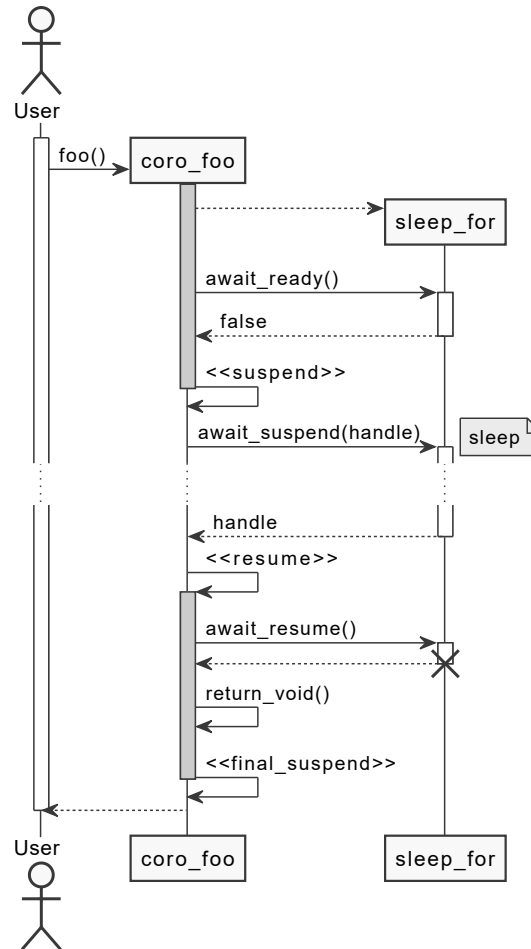
```
template<typename D>
struct sleep_for {
    D duration;

    bool await_ready() const noexcept { return duration <= D::zero(); }

    std::coroutine_handle<> await_suspend(std::coroutine_handle<> h)
    {
        std::this_thread::sleep_for(duration);
        return h;    // resume h, but from the calling frame, not from ours
    }

    void await_resume() noexcept {}
};
```

Symmetric Control Transfer: Flow



Scaling to millions of concurrent coroutines

CALLING `h.resume()`

```
caller
└─ await_suspend
    └─ h.resume() ← grows
        └─ await_suspend
            └─ h.resume()
                └─ ...
```

Stack depth = length of the await chain

RETURNING `h`

```
caller
└─ await_suspend → return h
caller resumes h ← flat
└─ await_suspend → return h
caller resumes h ← still flat
└─ ...
```

Stack depth is constant

Scaling to millions of concurrent coroutines

CALLING `h.resume()`

```
caller
└─ await_suspend
    └─ h.resume() ← grows
        └─ await_suspend
            └─ h.resume()
                └─ ...
```

Stack depth = length of the await chain

RETURNING `h`

```
caller
└─ await_suspend → return h
caller resumes h ← flat
└─ await_suspend → return h
caller resumes h ← still flat
└─ ...
```

Stack depth is constant

Default to returning the handle. Reach for `void` only when you have genuinely handed the handle to somebody else.

The Frame Is Your Scratch Space

```
task<int> async_op()
{
    structawaiter {
        int result;           // lives in the coroutine frame
        // ...
    };
    co_return co_await awaiter{};
}
```

- The coroutine is suspended while the worker runs
 - its frame, **including this awaiter**, stays alive
 - per-operation state with no **extra** heap allocation (we borrow the frame we already paid for instead)
 - only one actor works at a time - **no access synchronization needed**

The Frame Is Your Scratch Space

```
task<int> async_op()
{
    struct awaiter {
        int result;           // lives in the coroutine frame
        // ...
    };
    co_return co_await awaiter{};
}
```

- The coroutine is suspended while the worker runs
 - its frame, **including this awaiter**, stays alive
 - per-operation state with no **extra** heap allocation (we borrow the frame we already paid for instead)
 - only one actor works at a time - **no access synchronization needed**

Callback-based APIs often require dynamic allocation and synchronization.

The Frame Is Your Scratch Space

```
task<int> async_op()
{
    struct awaiter {
        int result; // lives in the coroutine frame
        bool await_ready() noexcept { return false; }
        void await_suspend(std::coroutine_handle<> h)
        {
            std::thread{[this, h] {
                result = do_expensive_work(); // written from the worker thread
                h.resume(); // ... and only then resumed
            }}.detach(); // don't detach it in production ;-)
        }
        int await_resume() noexcept { return result; }
    };
    co_return co_await awaiter{};
}
```

THE SHARP EDGES

The Awaiter Lives in the Frame You Just Released

`await_suspend` runs while the coroutine is suspended. The awaiter is usually a temporary inside that coroutine's frame. Resume the coroutine and it can run to completion, destroy its frame, and take your awaiter with it.

Two Ways to Hit It

DIRECTLY

```
void await_suspend(std::coroutine_handle<> h)
{
    h.resume();
    log(duration_);    // 💀
}
```

- The coroutine ran to completion inside `resume()`
- Frame freed, `duration_` is freed memory

Two Ways to Hit It

DIRECTLY

```
void await_suspend(std::coroutine_handle<> h)
{
    h.resume();
    log(duration_);    // 💀
}
```

- The coroutine ran to completion inside `resume()`
- Frame freed, `duration_` is freed memory

BY HANDING THE HANDLE AWAY

```
void await_suspend(std::coroutine_handle<> h)
{
    pool_.post([h] { h.resume(); });
    log(duration_);    // 💀 also
}
```

- The pool may resume `h` on another thread
before `await_suspend` even returns

Two Ways to Hit It

DIRECTLY

```
void await_suspend(std::coroutine_handle<> h)
{
    h.resume();
    log(duration_);    // 💀
}
```

- The coroutine ran to completion inside `resume()`
- Frame freed, `duration_` is freed memory

BY HANDING THE HANDLE AWAY

```
void await_suspend(std::coroutine_handle<> h)
{
    pool_.post([h] { h.resume(); });
    log(duration_);    // 💀 also
}
```

- The pool may resume `h` on another thread **before `await_suspend` even returns**

After the line that resumes or hands off the handle, ***this may already be destroyed**, not after `await_suspend` returns.

The Safe Shape

```
void await_suspend(std::coroutine_handle<> h)
{
    auto d = duration_;           // ✓ copy what you still need into locals FIRST
    pool_.post([h] { h.resume(); });
    log(d);                       // a local - nothing to do with the dead frame
}
```

The Safe Shape

```
void await_suspend(std::coroutine_handle<> h)
{
    auto d = duration_;           // copy what you still need into locals FIRST
    pool_.post([h] { h.resume(); });
    log(d);                       // a local - nothing to do with the dead frame
}
```

Copy to locals before you let go. Locals live in **await_suspend**'s own stack frame, which the coroutine cannot destroy.

The Safe Shape

```
void await_suspend(std::coroutine_handle<> h)
{
    auto d = duration_;           // copy what you still need into locals FIRST
    pool_.post([h] { h.resume(); });
    log(d);                       // ✓ a local - nothing to do with the dead frame
}
```

This is also why `await_resume` exists as a separate function. It runs after the coroutine is back and its frame is provably alive, so it is the one safe place to read the awaiter after you have let go and hand out a result.

Resuming Exactly Once

- **Exactly one** resumer, per suspension: never two, never none
 - **two of them** re-enter a frame that is **still running**, and corrupt it
 - **none**, and the frame **leaks**, with everything it owns

```
// ⚠ two resumer, one coroutine  
void await_suspend(std::coroutine_handle<> h)  
{  
    io_.on_done([h] { h.resume(); });  
    timer_.on_fire([h] { h.resume(); });  
}
```

Resuming Exactly Once

- **Exactly one** resumer, per suspension: never two, never none
 - **two of them** re-enter a frame that is **still running**, and corrupt it
 - **none**, and the frame **leaks**, with everything it owns

```
// ⚠ two resumers, one coroutine
void await_suspend(std::coroutine_handle<> h)
{
    io_.on_done([h] { h.resume(); });
    timer_.on_fire([h] { h.resume(); });
}
```

Two possible resumers? Make one cancel, not resume.

Committed to Suspending

The completion may fire after `await_ready` said `false` but before `await_suspend` resumes.

Committed to Suspending

The completion may fire after `await_ready` said `false` but before `await_suspend` resumes.

```
// a real question, with a real answer
bool await_ready() { return ev_.is_set(); }

// ...but it may have been set since
bool await_suspend(std::coroutine_handle<> h)
{
    return ev_.try_enqueue(h);    // false if already set
}
```

Committed to Suspending

The completion may fire after `await_ready` said `false` but before `await_suspend` resumes.

```
// a real question, with a real answer
bool await_ready() { return ev_.is_set(); }

// ...but it may have been set since
bool await_suspend(std::coroutine_handle<> h)
{
    return ev_.try_enqueue(h);    // false if already set
}
```

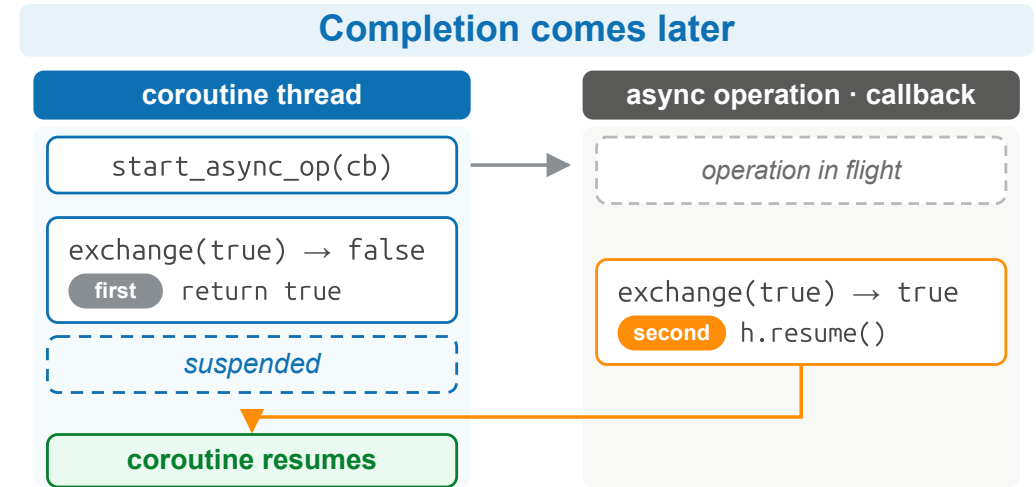
A void return is the one shape that cannot close it.

Claiming the Resumption

```
struct safe_async_waiter {  
    std::atomic<bool> claimed{false};  
  
    bool await_ready() noexcept { return false; }  
  
    bool await_suspend(  
        std::coroutine_handle<> h) noexcept  
    {  
        start_async_op([this, h] {  
            if (claimed.exchange(true))  
                h.resume();    // we are second  
        });  
        // work left after launch: hence the claim  
        return !claimed.exchange(true);  
    }  
  
    void await_resume() noexcept {}  
};
```

Claiming the Resumption

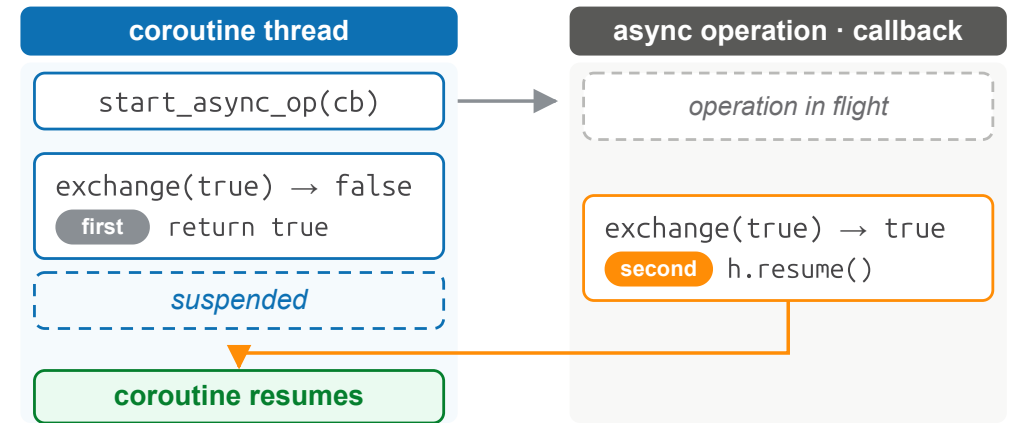
```
struct safe_async_waiter {  
    std::atomic<bool> claimed{false};  
  
    bool await_ready() noexcept { return false; }  
  
    bool await_suspend(  
        std::coroutine_handle<> h) noexcept  
    {  
        start_async_op([this, h] {  
            if (claimed.exchange(true))  
                h.resume();    // we are second  
        });  
        // work left after launch: hence the claim  
        return !claimed.exchange(true);  
    }  
  
    void await_resume() noexcept {}  
};
```



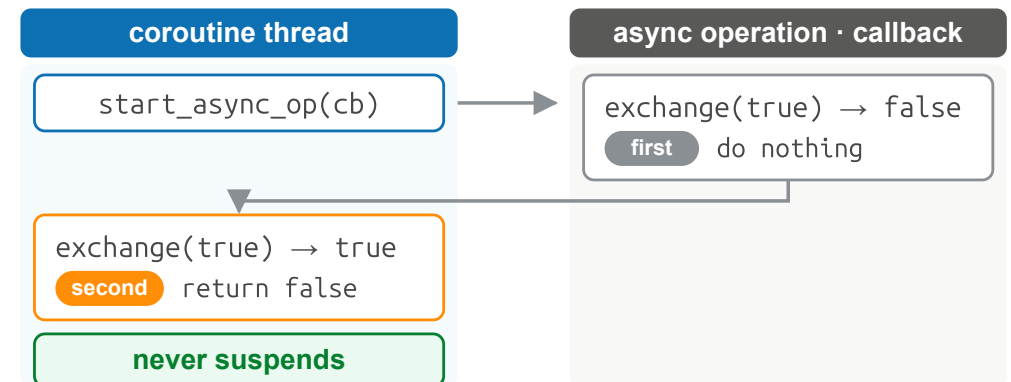
Claiming the Resumption

```
struct safe_async_waiter {  
    std::atomic<bool> claimed{false};  
  
    bool await_ready() noexcept { return false; }  
  
    bool await_suspend(  
        std::coroutine_handle<> h) noexcept  
    {  
        start_async_op([this, h] {  
            if (claimed.exchange(true))  
                h.resume();    // we are second  
        });  
        // work left after launch: hence the claim  
        return !claimed.exchange(true);  
    }  
  
    void await_resume() noexcept {}  
};
```

Completion comes later



Completion comes inline (same thread)



Never Destroy From an Awaiter

- A frame is destroyed exactly once too, but that is the **owner's** call
- Usually a **task<T>** holding a handle by RAII
 - destroy it here and the owner destroys it **again**
- An awaiter is a guest in that frame, never its owner

```
// ⚠ not yours to destroy
void await_suspend(
    std::coroutine_handle<> h)
{
    if (cancelled_)
        h.destroy(); // task<T> will too
}
```

Never Destroy From an Awaiter

- A frame is destroyed exactly once too, but that is the **owner's** call
- Usually a **task<T>** holding a handle by RAII
 - destroy it here and the owner destroys it **again**
- An awaiter is a guest in that frame, never its owner

```
//      not yours to destroy
void await_suspend(
    std::coroutine_handle<> h)
{
    if (cancelled_)
        h.destroy();    //      task<T> will too
}
```

Reach for your framework's cancellation instead.

Pass Parameters By Value

- A temporary bound to a **reference parameter** dies at the end of the caller's full-expression
- That is *before* the coroutine is ever resumed
- **`greet(std::string n)`** copies it into the frame instead, and the frame is already paid for

```
//      reference parameter
task<void> greet(const std::string& n)
{
    co_await sleep_for(1s);
    std::cout << n;    //      dangles
}

greet("hello");    // temporary dies at this ;
```

Pass Parameters By Value

- A temporary bound to a **reference parameter** dies at the end of the caller's full-expression
- That is *before* the coroutine is ever resumed
- **`greet(std::string n)`** copies it into the frame instead, and the frame is already paid for

```
//      reference parameter
task<void> greet(const std::string& n)
{
    co_await sleep_for(1s);
    std::cout << n;    //      dangles
}

greet("hello");    // temporary dies at this ;
```

A lazy coroutine has not run a single line when that `;` is reached. The body starts later, by which time the temporary is long gone.

No Locks Across a Suspension

- A coroutine may resume on a **different thread**
- **std::mutex** must be unlocked by the thread that locked it
- **lock_guard** lives in the frame, so it unlocks wherever the coroutine ends up
- Same for **thread_local**, and anything else thread-affine

```
//      held across the suspension
task<void> update(int v)
{
    std::lock_guard lk{mtx_};
    co_await async_write(v);
} //      unlock() on another thread
```

No Locks Across a Suspension

- A coroutine may resume on a **different thread**
- **std::mutex** must be unlocked by the thread that locked it
- **lock_guard** lives in the frame, so it unlocks wherever the coroutine ends up
- Same for **thread_local**, and anything else thread-affine

```
//      held across the suspension
task<void> update(int v)
{
    std::lock_guard lk{mtx_};
    co_await async_write(v);
} //      unlock() on another thread
```

A suspension point is a hole in time, and possibly a change of thread. Nothing thread-affine and nothing lifetime-fragile may cross it.

The Awaiter Temporary Outlives the Suspension

The awaiter might be a temporary, and the coroutine suspends in the middle of it.

The Awaiter Temporary Outlives the Suspension

The awaiter might be a temporary, and the coroutine suspends in the middle of it.

```
task<void> f()
{
    int n = co_await async_read(fd, buf);    // a temporary awaiter, suspended in the middle
}
```

The Awaiter Temporary Outlives the Suspension

The awaiter might be a temporary, and the coroutine suspends in the middle of it.

```
task<void> f()
{
    int n = co_await async_read(fd, buf);    // a temporary awaiter, suspended in the middle
}
```

A temporary lives to the end of its full-expression, and `co_await` is not over until `await_resume()` returns.

BRIDGING A C API

The API You Actually Have

```
typedef void (*read_cb)(int status, size_t n, void* user_data);  
int lib_read_async(lib_handle* h, char* buf, size_t len, read_cb cb, void* user_data);
```

The API You Actually Have

```
typedef void (*read_cb)(int status, size_t n, void* user_data);  
int lib_read_async(lib_handle* h, char* buf, size_t len, read_cb cb, void* user_data);
```

- A function pointer
 - no templates
 - no **std::function**
 - no lambdas
- A **void* user_data** echoed back to the callback

The API You Actually Have

```
typedef void (*read_cb)(int status, size_t n, void* user_data);  
int lib_read_async(lib_handle* h, char* buf, size_t len, read_cb cb, void* user_data);
```

- A function pointer
 - no templates
 - no **std::function**
 - no lambdas
- A **void*** **user_data** echoed back to the callback

Written in 2009, ships in production, and is not going to change.

Bridging It

```
class lib_read {
    lib_handle* h_;
    std::span<char> buf_;
    int status_ = 0;
    std::size_t n_ = 0;
    std::coroutine_handle<> waiter_;

    static void on_complete(int status,
                           std::size_t n,
                           void* user_data)
    {
        auto& self = *static_cast<lib_read*>(user_data);
        self.status_ = status;
        self.n_ = n;
        self.waiter_.resume();
    }
}
```

Bridging It

```
class lib_read {
    lib_handle* h_;
    std::span<char> buf_;
    int status_ = 0;
    std::size_t n_ = 0;
    std::coroutine_handle<> waiter_;

    static void on_complete(int status,
                           std::size_t n,
                           void* user_data)
    {
        auto& self = *static_cast<lib_read*>(user_data);
        self.status_ = status;
        self.n_ = n;
        self.waiter_.resume();
    }
}
```

```
public:
    lib_read(lib_handle* h, std::span<char> buf) noexcept :
        h_{h}, buf_{buf}
    {}

    bool await_ready() const noexcept { return false; }

    void await_suspend(std::coroutine_handle<> h)
    {
        waiter_ = h;
        lib_read_async(h_, buf_.data(), buf_.size(),
                       &on_complete, this);
    }

    std::size_t await_resume() const
    {
        if (status_ != 0) throw lib_error{status_};
        return n_;
    }
};
```

Two Ways a Real C API Will Betray You

IT FAILS WITHOUT CALLING BACK

```
int rc = lib_read_async(...);  
if (rc != 0) { /* no callback ever */ }
```

- The coroutine is suspended and **nobody will ever resume it**
- It leaks, silently, along with everything it owns

```
public:  
    lib_read(lib_handle* h, std::span<char> buf) noexcept :  
        h_{h}, buf_{buf}  
    {}  
  
    bool await_ready() const noexcept { return false; }  
  
    void await_suspend(std::coroutine_handle<> h)  
    {  
        waiter_ = h;  
        lib_read_async(h_, buf_.data(), buf_.size(),  
                        &on_complete, this);  
    }  
  
    std::size_t await_resume() const  
    {  
        if (status_ != 0) throw lib_error{status_};  
        return n_;  
    }  
};
```

We need to fix that.

Two Ways a Real C API Will Betray You

IT FAILS WITHOUT CALLING BACK

```
int rc = lib_read_async(...);  
if (rc != 0) { /* no callback ever */ }
```

- The coroutine is suspended and **nobody will ever resume it**
- It leaks, silently, along with everything it owns

```
public:  
    lib_read(lib_handle* h, std::span<char> buf) noexcept :  
        h_{h}, buf_{buf}  
    {}  
  
    bool await_ready() const noexcept { return false; }  
  
    bool await_suspend(std::coroutine_handle<> h)  
    {  
        waiter_ = h;  
        int rc = lib_read_async(h_, buf_.data(), buf_.size(),  
                                &on_complete, this);  
  
        if (rc != 0) {  
            status_ = rc;  
            // resume inline, report via await_resume  
            return false;  
        }  
        return true; // scheduled - stay suspended  
    }  
  
    std::size_t await_resume() const  
    {  
        if (status_ != 0) throw lib_error{status_};  
        return n_;  
    }  
};
```

Two Ways a Real C API Will Betray You

IT COMPLETES SYNCHRONOUSLY

- `lib_read_async` may call `on_complete` **before it returns**, on this thread
 - e.g. the data was already in the buffer
- The coroutine resumes, finishes, and the frame (and **`*this`**) is gone while we are still inside `await_suspend`

```
public:
    lib_read(lib_handle* h, std::span<char> buf) noexcept :
        h_{h}, buf_{buf}
    {}

    bool await_ready() const noexcept { return false; }

    bool await_suspend(std::coroutine_handle<> h)
    {
        waiter_ = h;
        int rc = lib_read_async(h_, buf_.data(), buf_.size(),
                                &on_complete, this);

        if (rc != 0) {
            status_ = rc;
            // resume inline, report via await_resume
            return false;
        }
        return true; // scheduled - stay suspended
    }

    std::size_t await_resume() const
    {
        if (status_ != 0) throw lib_error{status_};
        return n_;
    }
};
```

Two Ways a Real C API Will Betray You

IT COMPLETES SYNCHRONOUSLY

- `lib_read_async` may call `on_complete` **before it returns**, on this thread
 - e.g. the data was already in the buffer
- The coroutine resumes, finishes, and the frame (and ***this**) is gone while we are still inside `await_suspend`

```
public:
    lib_read(lib_handle* h, std::span<char> buf) noexcept :
        h_{h}, buf_{buf}
    {}

    bool await_ready() const noexcept { return false; }

    bool await_suspend(std::coroutine_handle<> h)
    {
        waiter_ = h;
        int rc = lib_read_async(h_, buf_.data(), buf_.size(),
                               &on_complete, this);

        if (rc != 0) {
            status_ = rc;
            // resume inline, report via await_resume
            return false;
        }
        return true; // scheduled - stay suspended
    }

    std::size_t await_resume() const
    {
```

Safe - because we touched nothing after the call.

Two Ways a Real C API Will Betray You

IT COMPLETES SYNCHRONOUSLY

- `lib_read_async` may call `on_complete` **before it returns**, on this thread
 - e.g. the data was already in the buffer
- The coroutine resumes, finishes, and the frame (and ***this**) is gone while we are still inside `await_suspend`

```
public:
    lib_read(lib_handle* h, std::span<char> buf) noexcept :
        h_{h}, buf_{buf}
    {}

    bool await_ready() const noexcept { return false; }

    bool await_suspend(std::coroutine_handle<> h)
    {
        waiter_ = h;
        int rc = lib_read_async(h_, buf_.data(), buf_.size(),
                               &on_complete, this);

        if (rc != 0) {
            status_ = rc;
            // resume inline, report via await_resume
            return false;
        }
        return true; // scheduled - stay suspended
    }
```

The cost is a nested resume, on the library's stack, inside its callback. Flatten with the **claimed** handshake from earlier when that matters.

TYPES YOU DO NOT OWN

Awaitable vs Awaiter

An awaitable is a type you are allowed to write `co_await` on *in a given coroutine*.

Awaitable vs Awaiter

An awaitable is a type you are allowed to write `co_await` on *in a given coroutine*.

An awaiter is a type implementing the three functions that a `co_await` expression actually calls: `await_ready`, `await_suspend`, `await_resume`.

Awaitable vs Awaiter

An awaitable is a type you are allowed to write `co_await` on *in a given coroutine*.

An awaiter is a type implementing the three functions that a `co_await` expression actually calls: `await_ready`, `await_suspend`, `await_resume`.

The same type can be both an awaitable and an awaiter type.

Sometimes an Awaitable Should Not Be an Awaiter

```
template<typename T>
struct [[nodiscard]] task {
    bool await_ready();
    std::coroutine_handle<> await_suspend(std::coroutine_handle<>);
    T await_resume();
    // ...
};
```

Sometimes an Awaitable Should Not Be an Awaiter

```
template<typename T>
struct [[nodiscard]] task {
    bool await_ready();
    std::coroutine_handle<> await_suspend(std::coroutine_handle<>);
    T await_resume();
    // ...
};
```

```
task<int> t = foo();
t.get();
t.await_resume();    //    why can a user call this?
```

Sometimes an Awaitable Should Not Be an Awaiter

```
template<typename T>
struct [[nodiscard]] task {
    bool await_ready();
    std::coroutine_handle<> await_suspend(std::coroutine_handle<>);
    T await_resume();
    // ...
};
```

```
task<int> t = foo();
t.get();
t.await_resume();    //      why can a user call this?
```

Either the awaiter interface is something you do not want public (`task<T>`), or the type is something you cannot modify at all (`std::future<T>`, `std::chrono::duration`).

operator co_await

A unary operator that turns an awaitable into an awaiter. The awaiter stays private. The awaitable is the public API.

operator co_await

A unary operator that turns an awaitable into an awaiter. The awaiter stays private. The awaitable is the public API.

MEMBER FORM

```
template<typename T>
class task {
    struct awaiter { /* the 3 functions */ };
public:
    auto operator co_await() &&
    {
        return awaiter{coro_.get()};
    }
};
```

Your type, hidden implementation

operator co_await

A unary operator that turns an awaitable into an awaiter. The awaiter stays private. The awaitable is the public API.

MEMBER FORM

```
template<typename T>
class task {
    struct awaiter { /* the 3 functions */ };
public:
    auto operator co_await() &&
    {
        return awaiter{coro_.get()};
    }
};
```

Your type, hidden implementation

FREE FUNCTION FORM

```
template<typename T>
auto operator co_await(thirdParty<T>& awaitable)
{
    struct awaiter {
        thirdParty<T>& a;
        bool await_ready();
        void await_suspend(std::coroutine_handle<> h);
        T await_resume();
    };
    return awaiter{awaitable};
}
```

Somebody else's type, untouched

How the Compiler Finds the Awaiter

- 1 `operator co_await` - member and free resolved as one overload set
- 2 No viable candidate: `the type is used directly` as the awaiter

How the Compiler Finds the Awaiter

- 1 `operator co_await` - member and free resolved as one overload set
- 2 No viable candidate: **the type is used directly** as the awaiter

```
template<typename T>
decltype(auto) get_awaiter(T&& awaitable)
{
    if constexpr(requires { std::forward<T>(awaitable).operator co_await(); } ||
                 requires { operator co_await(std::forward<T>(awaitable)); })
        return best_overload_of_operator_co_await(std::forward<T>(awaitable));
    else
        return std::forward<T>(awaitable);
}
```

Example: Making `std::chrono::duration` Awaitable

```
template<class Rep, class Period>
auto operator co_await(const std::chrono::duration<Rep, Period>& d)
{
    structawaiter {
        const std::chrono::duration<Rep, Period>& duration;

    };
    returnawaiter{d};
}
```

```
task<void> foo()
{
    co_await 1s;    // it works
}
```

Example: Making `std::chrono::duration` Awaitable

```
template<class Rep, class Period>
auto operator co_await(const std::chrono::duration<Rep, Period>& d)
{
    structawaiter {
        const std::chrono::duration<Rep, Period>& duration;

        bool await_ready() const noexcept
        {
            return duration <= std::chrono::duration<Rep, Period>::zero();
        }
        std::coroutine_handle<> await_suspend(std::coroutine_handle<> h) const
        {
            std::this_thread::sleep_for(duration);
            return h; // symmetric transfer
        }
        static void await_resume() noexcept {}
    };
    returnawaiter{d};
}
```

```
task<void> foo()
{
    co_await 1s; // it works
}
```

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

- It **collides** with every other library that had the same clever idea
 - two headers
 - one operator
 - one ill-formed program

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

- It **collides** with every other library that had the same clever idea
 - two headers
 - one operator
 - one ill-formed program

Neither library is wrong.

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

NO GOOD PLACE TO PUT IT ANYWAY

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

NO GOOD PLACE TO PUT IT ANYWAY

- Put it in your *own namespace* and **ADL cannot find it**
 - the associated namespaces are the *type's*, not yours

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

NO GOOD PLACE TO PUT IT ANYWAY

- Put it in your *own namespace* and **ADL cannot find it**
 - the associated namespaces are the *type's*, not yours
- Put it in *namespace std* and it is **undefined behavior**

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

PREFER INSTEAD

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

PREFER INSTEAD

- A **dedicated wrapper**
 - `my_namespace::sleep_for(1s)` rather than `co_await 1s`

Do Not Actually Do That

Never provide a global operator `co_await` for a type you do not own, such as `std::future<T>` or `std::chrono::duration`.

PREFER INSTEAD

- A **dedicated wrapper**
 - `my_namespace::sleep_for(1s)` rather than `co_await 1s`
- **Promise-specific support** via `await_transform`
 - active only inside your own coroutine type

SCOPING IT: `await_transform`

Per-Promise, Not Global

If a coroutine's promise type defines `await_transform`, the compiler rewrites every `co_await expr` in that coroutine's body into `co_await promise.await_transform(expr)`.

Per-Promise, Not Global

If a coroutine's promise type defines `await_transform`, the compiler rewrites every `co_await expr` in that coroutine's body into `co_await promise.await_transform(expr)`.

The scope of the customization is one coroutine type, chosen by the author of that coroutine type. Nobody else's coroutines are affected, and there is nothing to collide with.

Generally and Contextually Awaitables

Generally Awaitable is a type that supports the **operator** **co_await** in a coroutine context whose promise type does not have an **await_transform** member.

Generally and Contextually Awaitables

Generally Awaitable is a type that supports the **operator `co_await`** in a coroutine context whose promise type does not have an **`await_transform`** member.

Contextually Awaitable is a type that only supports the **operator `co_await`** in the context of certain types of coroutines due to the presence of an **`await_transform`** member function in the coroutine's promise type.

Use Case 1: Scope a Behavior to One Coroutine Type

```
template<typename T>
struct task {
    struct promise_type {
        template<class Rep, class Period>
        auto await_transform(std::chrono::duration<Rep, Period> d)
        {
            structawaiter { /* ... the sleep logic ... */ };
            returnawaiter{d};
        }
    };

    template<typenameAwaitable> // let everything else through
    decltype(auto) await_transform(Awaitable&& a)
    {
        returnstd::forward<Awaitable>(a);
    }
};
```

```
other_task<void> boo() // a coroutine type that does not opt in
{
    co_await1s; // error: no member named 'await_ready' in 'std::chrono::duration<...>'
}
```

Use Case 1: Scope a Behavior to One Coroutine Type

```
template<typename T>
struct task {
    struct promise_type {
        template<class Rep, class Period>
        auto await_transform(std::chrono::duration<Rep, Period> d)
        {
            structawaiter { /* ... the sleep logic ... */ };
            returnawaiter{d};
        }

        template<typenameAwaitable> // let everything else through
        decltype(auto) await_transform(Awaitable&& a)
        {
            returnstd::forward<Awaitable>(a);
        }
    };
};
```

```
other_task<void> boo() // a coroutine type that does not opt in
{
    co await 1s; // error: no member named 'await ready' in 'std::chrono::duration<...>'
```

Don't forget the fallback.

Use Case 2: `await_transform` as a Prohibition

```
template<typename T>
class generator_promise {
    // ...
    void await_transform() = delete;    // exactly what <generator> declares
};
```

Use Case 2: `await_transform` as a Prohibition

```
template<typename T>
class generator_promise {
    // ...
    void await_transform() = delete;    // exactly what <generator> declares
};
```

```
std::generator<int> my_gen()
{
    co_await something;    // error, and that is the intent
    co_yield 42;
}
```

Use Case 2: `await_transform` as a Prohibition

```
template<typename T>
class generator_promise {
    // ...
    void await_transform() = delete;    // exactly what <generator> declares
};
```

```
std::generator<int> my_gen()
{
    co_await something;    // error, and that is the intent
    co_yield 42;
}
```

A synchronous generator has no mechanism to suspend on asynchronous work, so allowing `co_await` in one would produce a coroutine that cannot possibly work.

Use Case 3: Inject Context on Resume

```
template<typename T, typename Executor>
class executor_task_promise {
    Executor executor_;
public:
    template<typename Awaitable>
    auto await_transform(Awaitable&& awaitable)
    {
        // wrap so that after it completes, the coroutine resumes on executor_
        return resume_on(executor_, std::forward<Awaitable>(awaitable));
    }
};
```

Use Case 3: Inject Context on Resume

```
template<typename T, typename Executor>
class executor_task_promise {
    Executor executor_;
public:
    template<typename Awaitable>
    auto await_transform(Awaitable&& awaitable)
    {
        // wrap so that after it completes, the coroutine resumes on executor_
        return resume_on(executor_, std::forward<Awaitable>(awaitable));
    }
};
```

Every **co_await** in the coroutine body quietly round-trips through the executor on the way back to ensure thread affinity.

What resume_on Is Made Of

```
template<typename A>
task<await_result_t<A>> resume_on(executor& ex, A a)
{
    class awaiter {
        executor& ex_;
    public:
        explicit awaiter(executor& ex) noexcept : ex_{ex} {}
        bool await_ready() const noexcept { return false; }
        void await_suspend(std::coroutine_handle<> h) { ex_.post([h] { h.resume(); }); }
        void await_resume() const noexcept {}
    };

    auto r = co_await std::move(a);    // 1. await it
    co_await awaiter{ex};              // 2. hop back
    co_return r;                      // void has its own overload
}
```

All Three, In One Declaration

```
// [task.promise]: the whole of
// std::execution::task's await_transform
template<sender Sender>
auto await_transform(Sender&& sndr)
{
    if constexpr (same_as<inline_scheduler,
                        start_scheduler_type>)
        return as_awaitable(
            std::forward<Sender>(sndr), *this);
    else
        return as_awaitable(
            affine(std::forward<Sender>(sndr)), *this);
}
```

- **affine** puts the coroutine **back on its own scheduler** after every **co_await**
- **as_awaitable** turns the **sender** into the **awaiter**
- **sender** `Sender`, and **no fallback**: anything else does not compile
 - all generally awaitables are senders

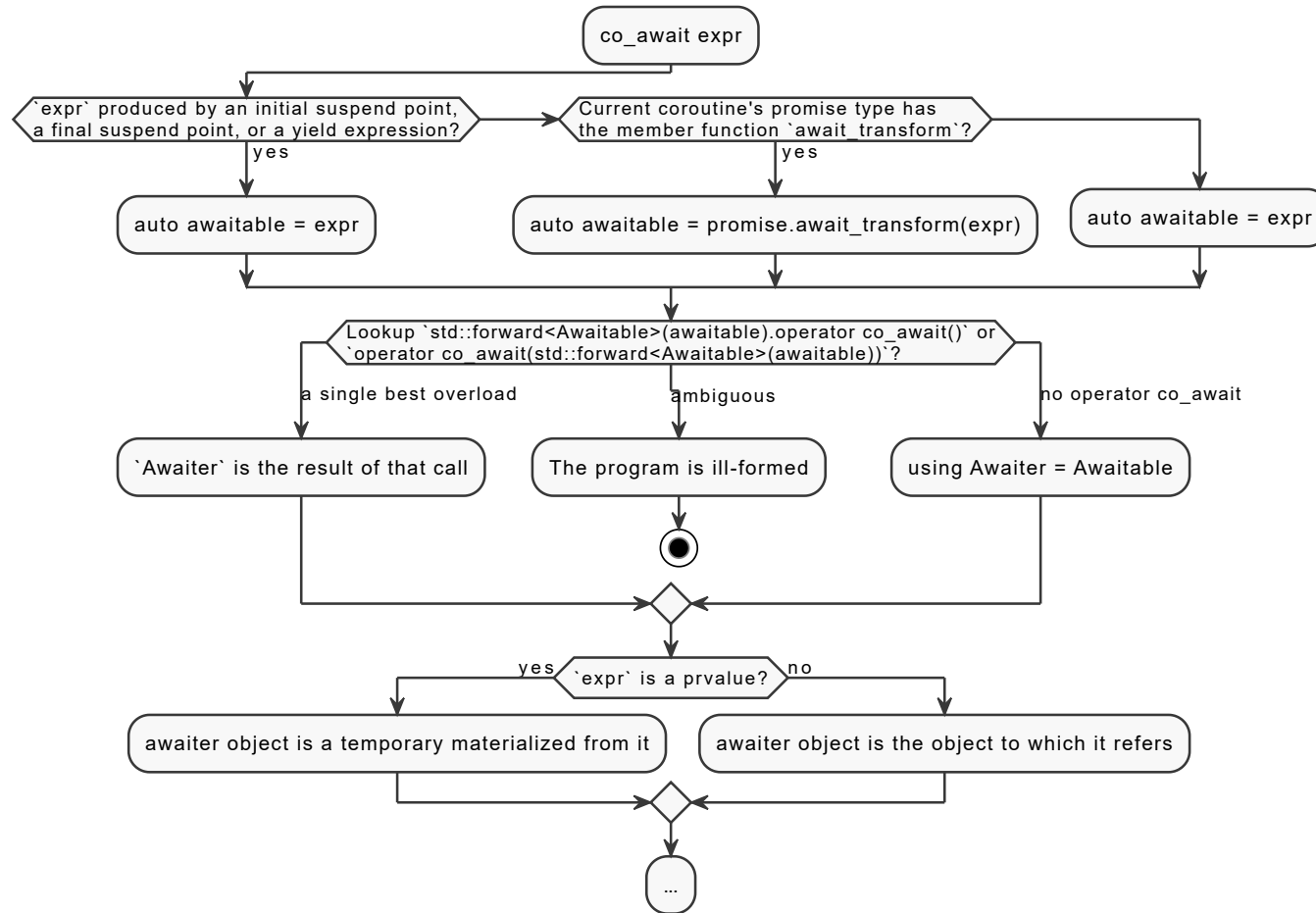
All Three, In One Declaration

```
// [task.promise]: the whole of
// std::execution::task's await_transform
template<sender Sender>
auto await_transform(Sender&& sndr)
{
    if constexpr (same_as<inline_scheduler,
                        start_scheduler_type>)
        return as_awaitable(
            std::forward<Sender>(sndr), *this);
    else
        return as_awaitable(
            affine(std::forward<Sender>(sndr)), *this);
}
```

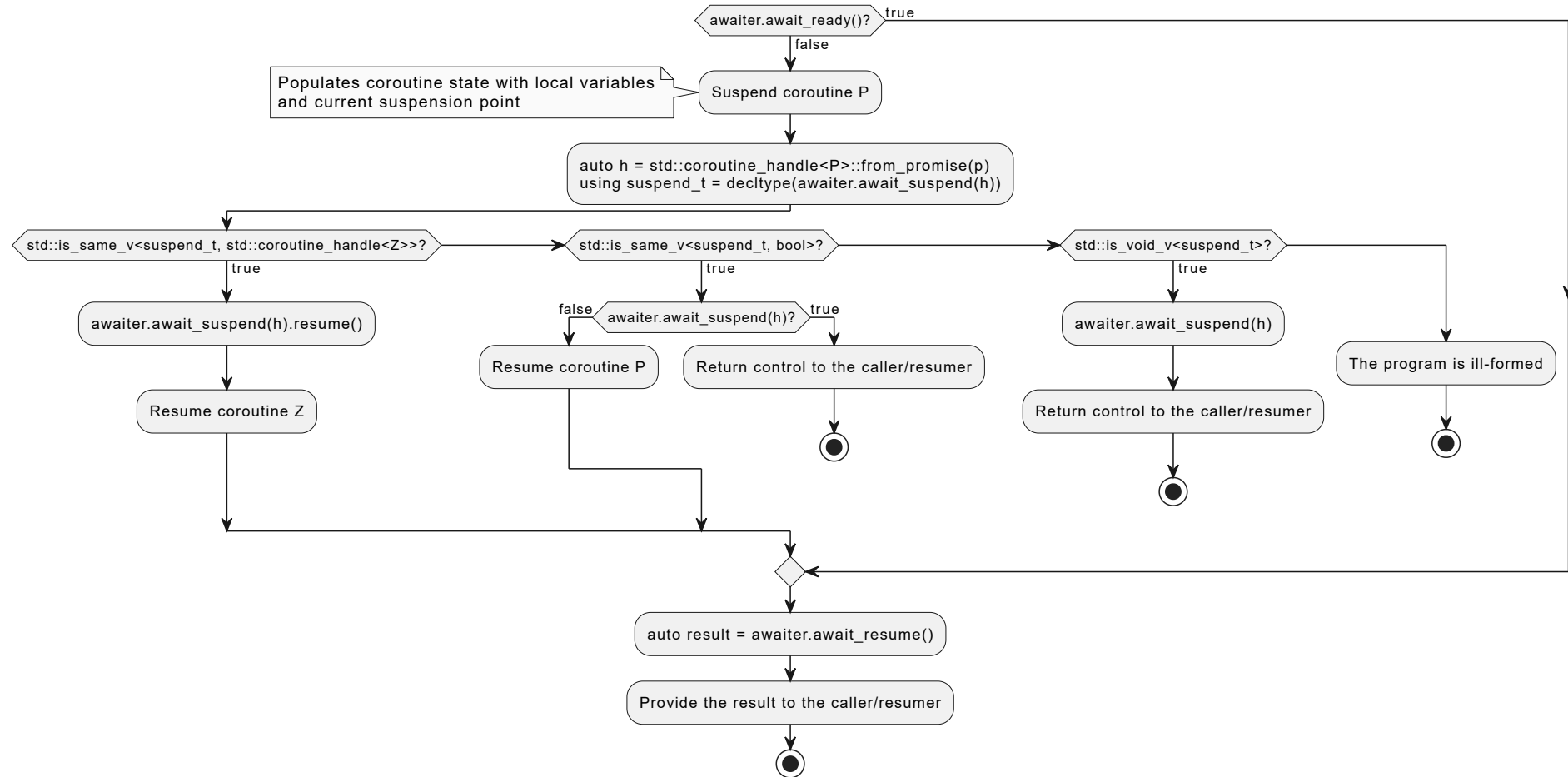
- **affine** puts the coroutine **back on its own scheduler** after every **co_await**
- **as_awaitable** turns the **sender** into the **awaiter**
- **sender** `Sender`, and **no fallback**: anything else does not compile
 - all generally awaitables are senders

The fallback is a default, not a law.

The Full Resolution



The Full Suspension and Resumption



ONE COROUTINE AWAITING ANOTHER

co_await task<T>

```
task<int> foo()
{
    co_return 42;
}

task<int> bar()
{
    const int i = co_await foo();
    co_return i + 23;
}
```

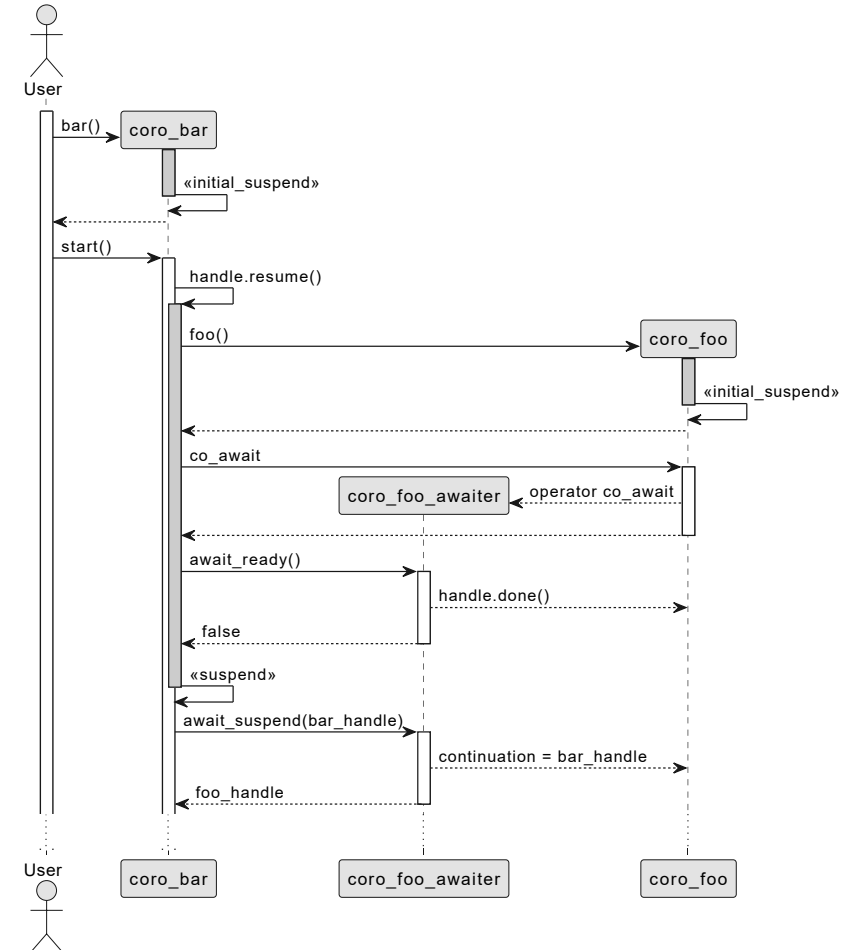
```
auto task = bar();
task.start();
std::cout << task.get() << "\n";
```

co_await task<T>

```
task<int> foo()
{
    co_return 42;
}

task<int> bar()
{
    const int i = co_await foo();
    co_return i + 23;
}
```

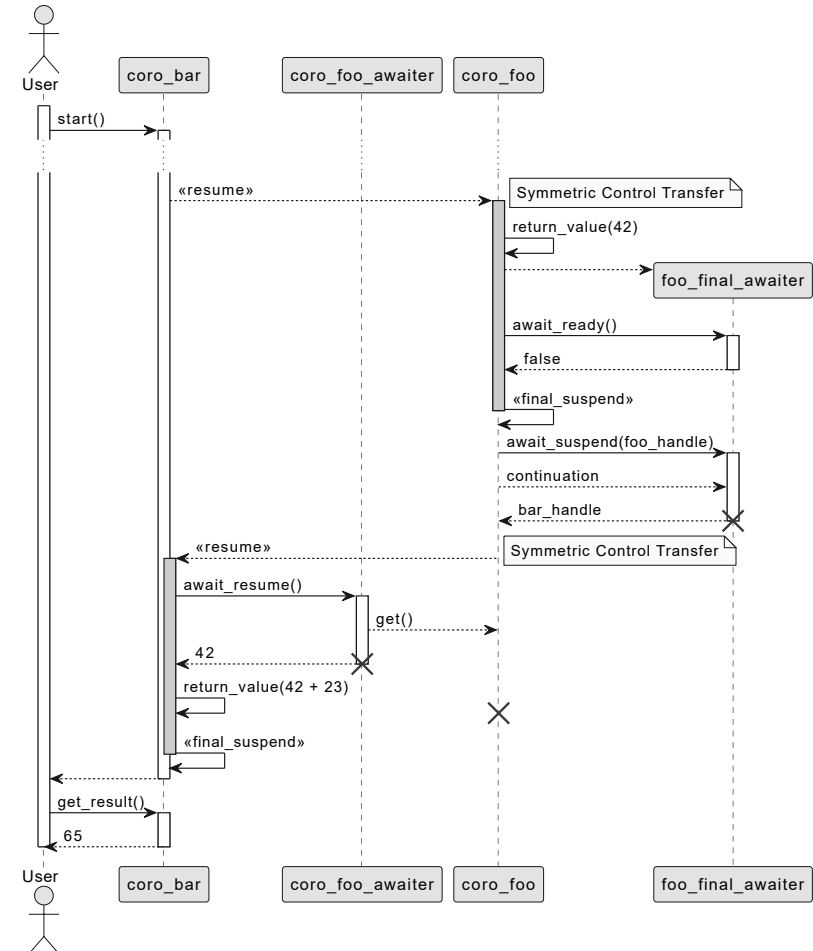
```
auto task = bar();
task.start();
std::cout << task.get() << "\n";
```



co_await task<T>

```
task<int> foo()  
{  
    co_return 42;  
}  
  
task<int> bar()  
{  
    const int i = co_await foo();  
    co_return i + 23;  
}
```

```
auto task = bar();  
task.start();  
std::cout << task.get() << "\n";
```



What That Awaiter Looks Like

```
template<typename T>
class task {
    struct awaiter {
        std::coroutine_handle<promise_type> handle;

        bool await_ready() const noexcept
        {
            return handle.done();                // already finished?
        }

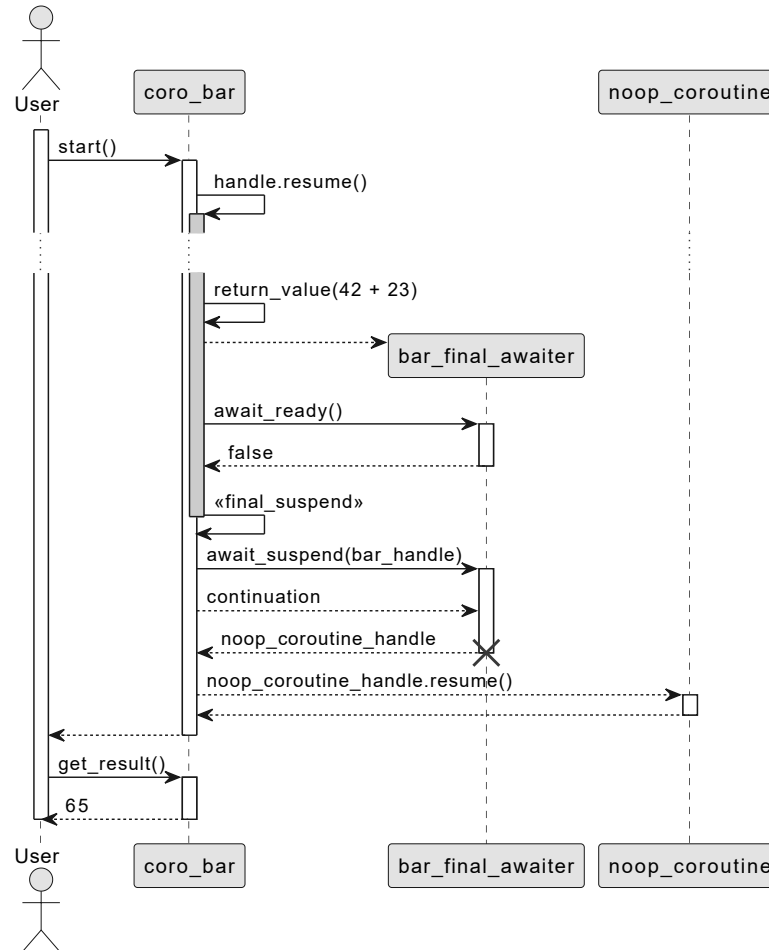
        std::coroutine_handle<> await_suspend(std::coroutine_handle<> caller) noexcept
        {
            handle.promise().continuation = caller;    // remember who to come back to
            return handle;                            // symmetric transfer: start foo
        }

        T await_resume() const
        {
            return handle.promise().get();            // foo's result, or its exception
        }
    };
    // ...
public:
    auto operator co_await() && noexcept { return awaiter{handle_.get()}; }
    // ...
};
```

The Chain Has to Stop Somewhere

```
struct final_awaiter : std::suspend_always {
    std::coroutine_handle<> await_suspend(std::coroutine_handle<promise_type> h) noexcept
    {
        if(auto k = h.promise().continuation)
            return k;                // resume whoever is waiting for us
        return std::noop_coroutine(); // nobody is: stop here, flat
    }
};
```

The Chain Has to Stop Somewhere



COOPERATIVE CANCELLATION

Short Intro To C++20 Cancellation Facilities

```
std::stop_source src;  
std::stop_token tok = src.get_token();  
  
std::stop_callback cb{tok, [] {  
    // runs on whichever thread calls  
    // request_stop() - or right here and now,  
    // if a stop was already requested  
}};  
  
src.request_stop();    // fires the callbacks
```

Short Intro To C++20 Cancellation Facilities

```
std::stop_source src;
std::stop_token tok = src.get_token();

std::stop_callback cb{tok, [] {
    // runs on whichever thread calls
    // request_stop() - or right here and now,
    // if a stop was already requested
}};

src.request_stop(); // fires the callbacks
```

- **std::stop_source** owns the state
 - **request_stop()** sets it **once**
- **std::stop_token** only *observes* it
 - copyable, cheap, requests nothing
 - can be polled with **st.stop_requested()**
- **std::stop_callback** registers a callable on a token, RAII

Long Awaits Ignore the World

```
task<void> do_work(std::stop_token st)
{
    while (!st.stop_requested()) {
        co_await sleep_for(60s);    //    cancelled 1 second in, and we still wait 59 more
        process_next_item();
    }
}
```

Long Awaits Ignore the World

```
task<void> do_work(std::stop_token st)
{
    while (!st.stop_requested()) {
        co_await sleep_for(60s);    // cancelled 1 second in, and we still wait 59 more
        process_next_item();
    }
}
```

Checking **stop_requested()** between awaits is not enough. An awaiter that ignores cancellation holds up the whole coroutine chain for as long as its operation takes.

What We Are Aiming At

THE I/O LAYER WE ASSUME

```
struct timer {  
    // per-wait state, owned by the awaiter  
    struct operation { /* ... */ };  
  
    // on_expiry(bool cancelled), exactly once  
    void arm(operation& state,  
             std::chrono::nanoseconds duration,  
             auto on_expiry);  
  
    // does not resume anything  
    // completes the operation early  
    void cancel(operation&);  
};
```

What We Are Aiming At

THE I/O LAYER WE ASSUME

```
struct timer {  
    // per-wait state, owned by the awaiter  
    struct operation { /* ... */ };  
  
    // on_expiry(bool cancelled), exactly once  
    void arm(operation& state,  
             std::chrono::nanoseconds duration,  
             auto on_expiry);  
  
    // does not resume anything  
    // completes the operation early  
    void cancel(operation&);  
};
```

WHAT WE WANT TO WRITE

```
task<void> do_work(std::stop_token st)  
{  
    using namespace std::chrono_literals;  
  
    // the condition *is* the await now  
    while (co_await cancellable_sleep_for{timer, 60s, st})  
        process_next_item(); // slept the full minute  
    // false = stop requested, and we are out at once  
}
```

What We Are Aiming At

THE I/O LAYER WE ASSUME

```
struct timer {  
    // per-wait state, owned by the awaiter  
    struct operation { /* ... */ };  
  
    // on_expiry(bool cancelled), exactly once  
    void arm(operation& state,  
             std::chrono::nanoseconds duration,  
             auto on_expiry);  
  
    // does not resume anything  
    // completes the operation early  
    void cancel(operation&);  
};
```

WHAT WE WANT TO WRITE

```
task<void> do_work(std::stop_token st)  
{  
    using namespace std::chrono_literals;  
  
    // the condition *is* the await now  
    while (co_await cancellable_sleep_for{timer, 60s, st})  
        process_next_item(); // slept the full minute  
    // false = stop requested, and we are out at once  
}
```

Waking the coroutine while the operation keeps running is not cancellation.
The request has to reach the I/O layer: Asio slots, `io_uring`, `CancelIoEx`.

A Cancellation-Aware Awaiter: The Contract

- 1 `await_ready`: if a stop is already requested, do not suspend at all

A Cancellation-Aware Awaiter: The Contract

- 1 **await_ready**: if a stop is already requested, do not suspend at all
- 2 **Register the stop_callback before arming the operation**
 - A stop request may arrive during setup and must not be missed

A Cancellation-Aware Awaiter: The Contract

- 1 **await_ready**: if a stop is already requested, do not suspend at all
- 2 **Register the stop_callback before arming the operation**
 - A stop request may arrive during setup and must not be missed
- 3 **The cancellation callback cancels the operation. It does not resume the coroutine**
 - Exactly one thing resumes: the operation's own completion

A Cancellation-Aware Awaiter: The Contract

- 1 **await_ready**: if a stop is already requested, do not suspend at all
- 2 **Register the stop_callback before arming the operation**
 - A stop request may arrive during setup and must not be missed
- 3 **The cancellation callback cancels the operation. It does not resume the coroutine**
 - Exactly one thing resumes: the operation's own completion
- 4 **Deregister in await_resume**
 - `~stop_callback` waits for a callback running on another thread
 - nothing touches the awaiter after this

A Cancellation-Aware Awaiter

```
template<typename Timer>
class cancellable_sleep_for {
    Timer& timer_;
    std::chrono::nanoseconds duration_;
    std::stop_token token_;
    Timer::operation op_{};
    bool cancelled_ = false;
    std::optional<std::stop_callback<
        std::function<void()>>> cb_;
public:
    cancellable_sleep_for(Timer& t,
                          std::chrono::nanoseconds d,
                          std::stop_token st):
        timer_{t}, duration_{d}, token_{std::move(st)} {}

    bool await_ready() noexcept
    {
        if (token_.stop_requested()) {
            // cancelled early
            cancelled_ = true;
            return true;
        }
        return duration_ <= std::chrono::nanoseconds::zero();
    }
}
```

A Cancellation-Aware Awaiter

```
template<typename Timer>
class cancellable_sleep_for {
    Timer& timer_;
    std::chrono::nanoseconds duration_;
    std::stop_token token_;
    Timer::operation op_{};
    bool cancelled_ = false;
    std::optional<std::stop_callback<
        std::function<void()>>> cb_;
public:
    cancellable_sleep_for(Timer& t,
                          std::chrono::nanoseconds d,
                          std::stop_token st):
        timer_{t}, duration_{d}, token_{std::move(st)} {}

    bool await_ready() noexcept
    {
        if (token_.stop_requested()) {
            // cancelled early
            cancelled_ = true;
            return true;
        }
        return duration_ <= std::chrono::nanoseconds::zero();
    }
};
```

```
void await_suspend(std::coroutine_handle<> h)
{
    // cancel only, never resume
    cb_.emplace(token_, [this] { timer_.cancel(op_); });

    // the sleep, and the only resumer: armed last
    timer_.arm(op_, duration_, [this, h](bool cancelled) {
        cancelled_ = cancelled;
        h.resume();
    });
    // touch nothing after this
}

bool await_resume() noexcept
{
    // ~stop_callback waits for any in-flight callback
    cb_.reset();
    return !cancelled_;
}
};
```

SUMMARY

You Will Write Fewer Coroutine Types

`std::execution::task<T>` lands in C++26, so the **`promise_type`** you never enjoyed writing is now somebody else's problem. You may already be there, if you use `cppcoro`, `libunifex`, or `Asio`.

You Will Write Fewer Coroutine Types

`std::execution::task<T>` lands in C++26, so the `promise_type` you never enjoyed writing is now somebody else's problem. You may already be there, if you use `cppcoro`, `libunifex`, or `Asio`.

You will still write `awaiters`. Somebody has to translate a callback API into a `co_await` expression, and in 2029 your vendor SDK will still take a function pointer.

Three Functions

THE PROTOCOL

```
struct awaiter {  
    // is the result already here?  
    bool await_ready();  
  
    // arrange to come back  
    auto await_suspend(std::coroutine_handle<>);  
  
    // what is the value?  
    T await_resume();  
};
```

Three Functions

THE PROTOCOL

```
struct awaiter {  
    // is the result already here?  
    bool await_ready();  
  
    // arrange to come back  
    auto await_suspend(std::coroutine_handle<>);  
  
    // what is the value?  
    T await_resume();  
};
```

THE API FROM MINUTE ONE

```
struct read_temp { // the bridge  
    sensor& s;  
    int value;  
    bool await_ready() { return false; }  
    void await_suspend(std::coroutine_handle<> h)  
    {  
        read_temperature(s, [this, h](int t) {  
            value = t; // results first  
            h.resume(); // resume last  
        });  
    }  
    int await_resume() { return value; }  
};  
  
task<void> monitor(sensor& s) // what we wanted  
{  
    int t = co_await read_temp{s};  
    log(t);  
}
```

Awaitable Is the API, Awaiter Is the Machinery

```
task<int> t = foo();  
t.await_resume();    //    nobody should be able to write this
```

Awaitable Is the API, Awaiter Is the Machinery

```
task<int> t = foo();  
t.await_resume();    // ⚠ nobody should be able to write this
```

- Your users **co_await** an **awaitable**. That is the whole public surface
- An **awaiter** is three functions that only the compiler should ever see and call
- **operator co_await** keeps them apart
- **await_transform** does it per coroutine type

Prefer Symmetric Transfer

```
std::coroutine_handle<> await_suspend(std::coroutine_handle<> h)
{
    other.continuation = h;
    return other_handle;    // tail call: this frame is already gone
}
```

Prefer Symmetric Transfer

```
std::coroutine_handle<> await_suspend(std::coroutine_handle<> h)
{
    other.continuation = h;
    return other_handle;    // tail call: this frame is already gone
}
```

- Calling **resume()** inside **await_suspend** nests it
- Nesting works perfectly until a loop does it ten thousand times
- Returning a handle transfers control
- Returning **bool** is the same idea for "the result was already here"

Prefer Symmetric Transfer

```
std::coroutine_handle<> await_suspend(std::coroutine_handle<> h)
{
    other.continuation = h;
    return other_handle;    // tail call: this frame is already gone
}
```

- Calling **resume()** inside **await_suspend** nests it
- Nesting works perfectly until a loop does it ten thousand times
- Returning a handle transfers control
- Returning **bool** is the same idea for "the result was already here"

The stack depth of a coroutine chain should be a property of the machine, not of how many times you happened to **co_await** in a loop.

Where to Go Next

MORE INFO

- [mp-coroutine](#) - `task`, `generator`, `sync_await`, `when_all` from my workshops, written to be [read](#)
- [Lewis Baker](#) on `operator co_await` and symmetric transfer
- [stdexec](#) - the `std::execution` reference implementation, on today's compilers

Where to Go Next

MORE INFO

- [mp-coroutine](#) - `task`, `generator`, `sync_await`, `when_all` from my workshops, written to be [read](#)
- [Lewis Baker](#) on `operator co_await` and symmetric transfer
- [stdexec](#) - the `std::execution` reference implementation, on today's compilers

WHERE DO YOU STAND?

- *"How modern is your C++?"*
- *"How solid are your C++ fundamentals?"*
- A [team health check](#), for leads sizing up a group



train-it.eu/self-checks

train IT

Questions?



The background is a solid yellow color. It is decorated with several black geometric shapes, primarily parallelograms and triangles, arranged in a pattern that suggests a 3D perspective or a stylized architectural design. These shapes are positioned around the edges and corners of the frame.

CAUTION
Programming
is addictive
(and too much fun)